DESIGN AND DEVELOP AN MQTT-BASED REMOTE HVAC MONITORING SYSTEM USING THINGSPEAK CLOUD. THE SYSTEM REMOTELY MONITORS HVAC PARAMETERS AND STORES SENSOR DATA ON CLOUD PLATFORMS FOR ANALYSIS AND CONTROL

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B – 24MER030

MOHAMMED SABEER S – 24MER031

PRAAVIN RAJAA G – 24MER039

PRASANNA T – 24MER042

DEVA PRIYAN R – 24MEL064

SATHISH KUMAR K – 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The MQTT-Based Remote HVAC Monitoring and Control System Using ThingSpeak Cloud is an IoT-based system designed to monitor and manage HVAC (Heating, Ventilation, and Air Conditioning) systems remotely. Sensors are used to measure important HVAC parameters such as temperature, humidity, air quality, pressure, and energy consumption. The collected sensor data is processed by a microcontroller and transmitted through the MQTT communication protocol to the cloud platform.

The ThingSpeak Cloud is used to store, visualize, and analyze the collected data in real time. Users can remotely access the HVAC system through an internet-connected device and monitor its operating conditions. Based on predefined conditions or threshold values, the system can also support remote control of HVAC equipment. This approach helps improve energy efficiency, system performance, fault detection, and user convenience while reducing the need for continuous manual monitoring.

2. Problem Statement

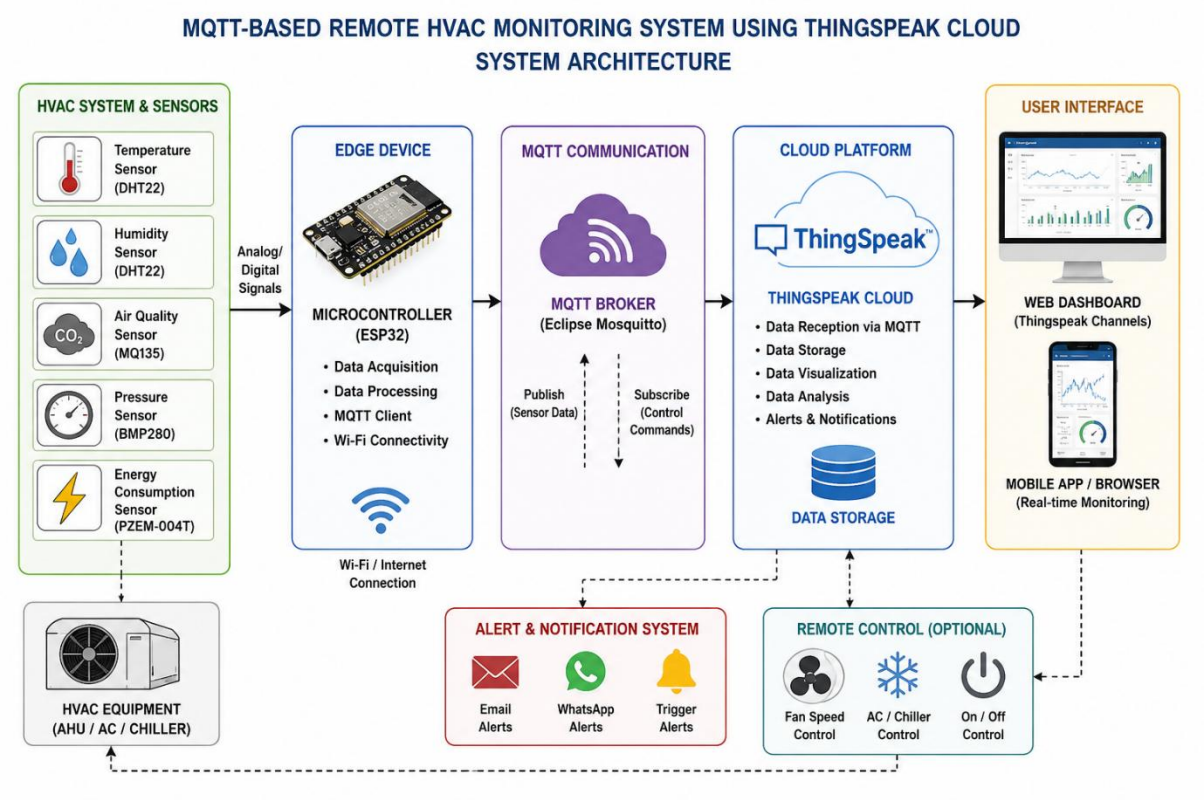
Traditional HVAC systems often require manual monitoring and control, making it difficult to continuously observe important parameters such as temperature, humidity, air quality, and energy consumption. Any abnormal operating condition or equipment fault may not be detected immediately, which can result in energy wastage, reduced system efficiency, increased maintenance costs, and discomfort to users. In addition, conventional systems generally lack centralized cloud-based data storage and real-time remote monitoring capabilities. Therefore, there is a need for an IoT-based remote HVAC monitoring system that can collect sensor data, transmit it reliably using the MQTT protocol, store and visualize the information on ThingSpeak Cloud, and enable users to monitor and control HVAC operation remotely.

3. Proposed Solution

The proposed system develops an IoT-based remote HVAC monitoring and control system using MQTT communication and ThingSpeak Cloud. Sensors are installed in the HVAC system to continuously measure parameters such as temperature, humidity, air quality, pressure, and energy consumption. A microcontroller collects and processes the sensor readings and publishes the data through the MQTT protocol over an internet connection.

The sensor data is transmitted to ThingSpeak Cloud, where it is stored, displayed through real-time graphs, and analyzed for performance monitoring. Predefined threshold values can be used to identify abnormal conditions and support automatic or remote control of HVAC equipment. Users can access the cloud dashboard from a computer or mobile device to monitor system conditions remotely. The proposed solution helps achieve real-time monitoring, efficient energy management, early fault detection, reduced maintenance requirements, and improved HVAC performance.

4. System Architecture

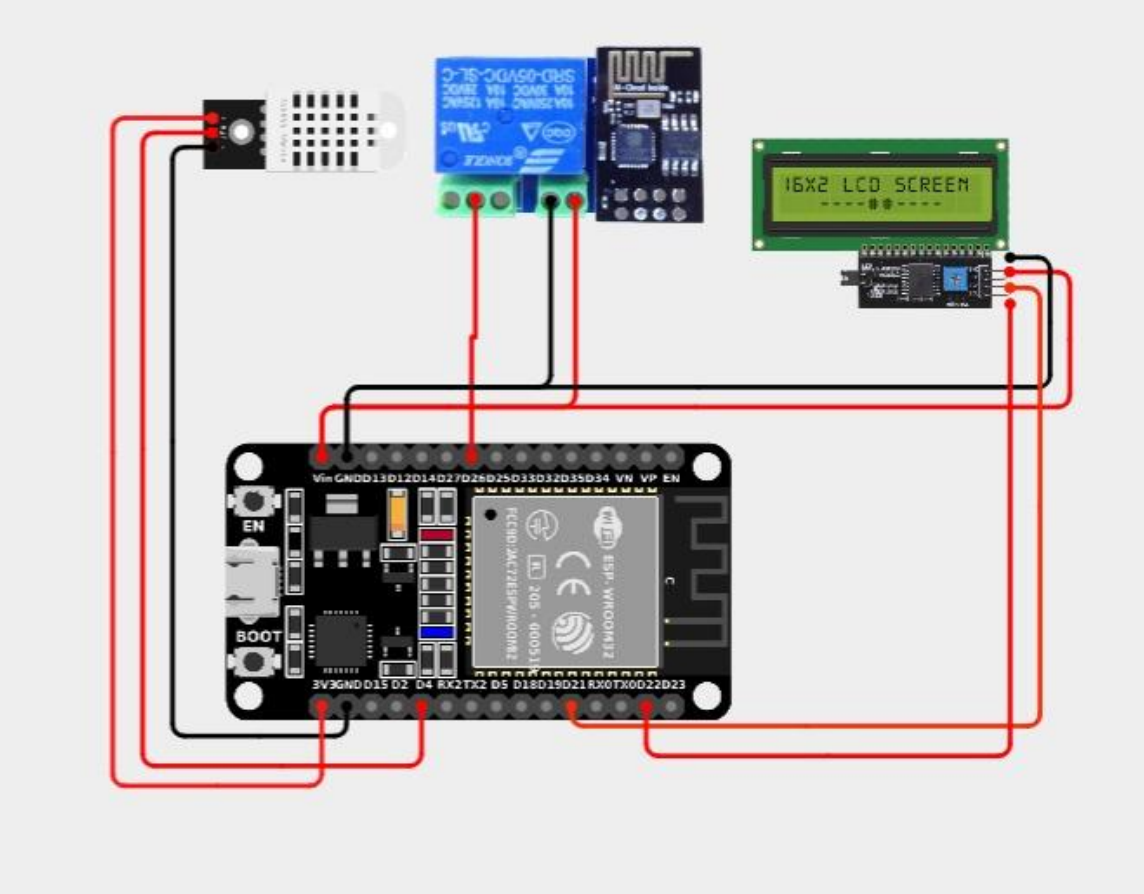


5. Circuit Table

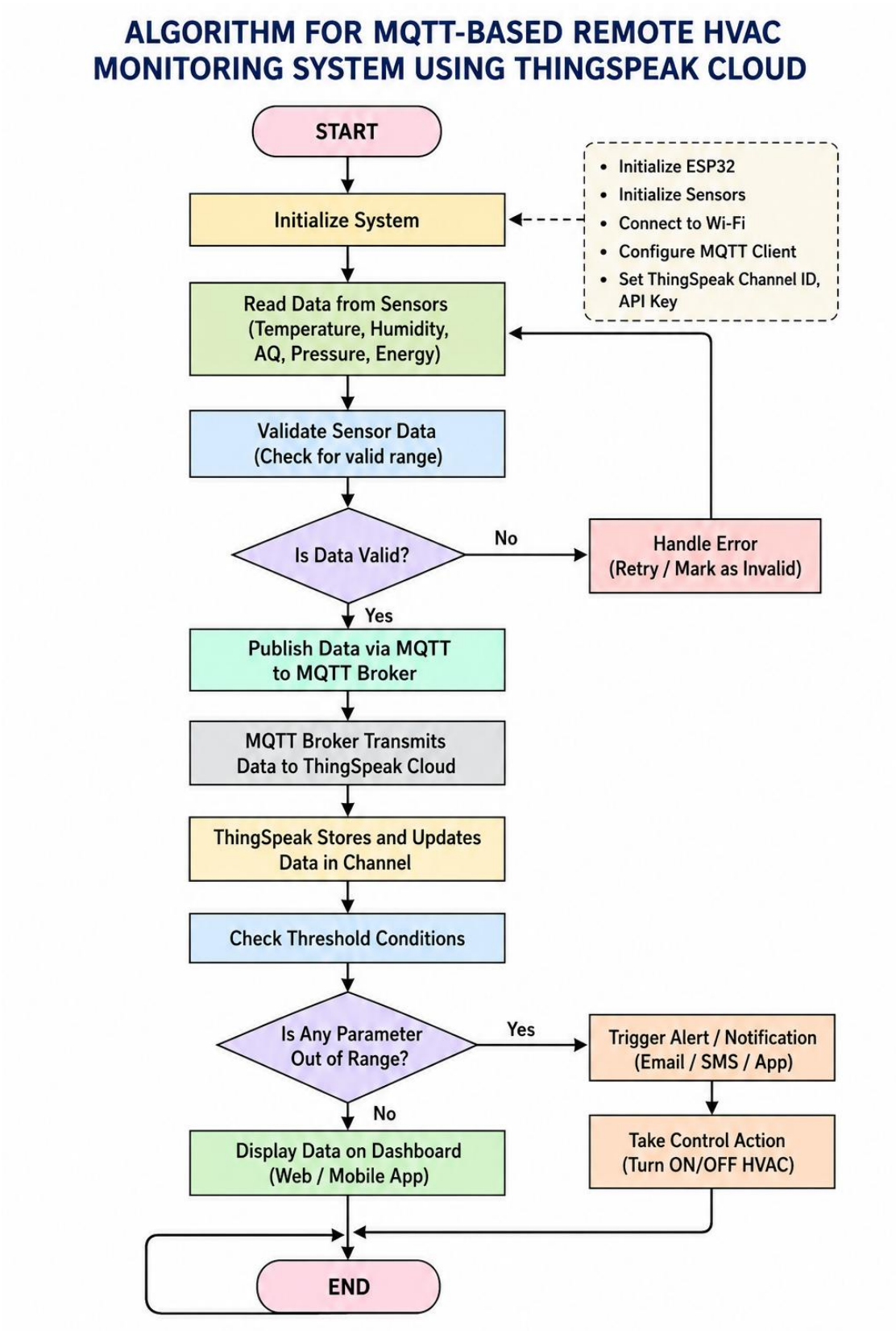
S.No.	Component	Pin/Terminal	Connected To	Purpose
1	ESP32	3.3V	Sensor VCC	Provides power to sensors
2	ESP32	GND	Sensor GND	Common ground connection
3	DHT22	DATA	GPIO 4	Measures temperature and humidity
4	MQ-135	AO	GPIO 34 (ADC)	Measures air-quality level
5	BMP280	SDA	GPIO 21	Measures atmospheric pressure
6	BMP280	SCL	GPIO 22	I²C clock communication
7	PZEM-004T	TX	ESP32 RX2 (GPIO 16)	Sends energy measurement data
8	PZEM-004T	RX	ESP32 TX2 (GPIO 17)	Receives communication commands

S.No.	Component	Pin/Terminal	Connected To	Purpose
9	Relay Module	IN	GPIO 26	Controls HVAC ON/OFF operation
10	Relay Module	VCC	5V	Relay power supply
11	Relay Module	GND	GND	Common ground
12	ESP32	Wi-Fi	Internet Router	Provides internet connectivity
13	ESP32	MQTT Client	MQTT Broker	Publishes sensor data
14	MQTT Broker	MQTT	ThingSpeak Cloud	Transfers data to cloud
15	ThingSpeak	Cloud Channel	Web/Mobile Dashboard	Stores and visualizes HVAC data

6. Circuit Design



7. Algorithm


ALGORITHM FOR MQTT-BASED REMOTE HVAC MONITORING SYSTEM USING THINGSPEAK CLOUD
START
Initialize System
Initialize ESP32
Initialize Sensors
Connect to Wi-Fi
Configure MQTT Client
Set ThingSpeak Channel ID, API Key
Read Data from Sensors (Temperature, Humidity, AQ, Pressure, Energy)
Validate Sensor Data (Check for valid range)
Is Data Valid?
No
Handle Error (Retry / Mark as Invalid)
Yes
Publish Data via MQTT to MQTT Broker
MQTT Broker Transmits Data to ThingSpeak Cloud
ThingSpeak Stores and Updates Data in Channel
Check Threshold Conditions
Is Any Parameter Out of Range?
Yes
Trigger Alert / Notification (Email / SMS / App)
Take Control Action (Turn ON/OFF HVAC)
No
Display Data on Dashboard (Web / Mobile App)
END

8. Coding

```
//
================================================================
===
// REMOTE HVAC MONITORING SYSTEM
// ESP32 + DHT22 + Relay + OLED + ThingSpeak
//
================================================================
===

#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

//
================================================================
===
// 1. WIFI DETAILS
//
================================================================
===

const char* WIFI_SSID = "kowsi";
const char* WIFI_PASSWORD = "kowsi @ 01";
```

```
//
================================================================
===
// 2. THINGSPEAK DETAILS
//
================================================================
===

// PUT YOUR THINGSPEAK WRITE API KEY HERE
const char* THINGSPEAK_API_KEY = "XPRJ6BREE0FXE4LV";

// ThingSpeak server
const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";

//
================================================================
===
// 3. DHT22 SENSOR
//
================================================================
===

#define DHT_PIN 4
#define DHT_TYPE DHT22

DHT dht(DHT_PIN, DHT_TYPE);
```

```
//
================================================================
===
// 4. RELAY
//
================================================================
===

#define RELAY_PIN 26

// Most relay modules are ACTIVE LOW
#define RELAY_ON LOW
#define RELAY_OFF HIGH

bool hvacState = false;

//
================================================================
===
// 5. OLED DISPLAY
//
================================================================
===

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define OLED_SDA 21
```

```
#define OLED_SCL 22

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

//
================================================================
===
// 6. TIMER
//
================================================================
===

unsigned long previousMillis = 0;

// ThingSpeak update interval
// 15 seconds
const unsigned long UPDATE_INTERVAL = 15000;

//
================================================================
===
// 7. CONNECT TO WIFI
```

```
//
================================================================
===

void connectWiFi()
{
  Serial.println();
  Serial.println("Connecting to WiFi...");

  WiFi.begin(
    WIFI_SSID,
    WIFI_PASSWORD
  );

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}
```

```
//
================================================================
===
// 8. OLED DISPLAY
//
================================================================
===

void updateOLED(
  float temperature,
  float humidity
)
{
  display.clearDisplay();

  display.setTextColor(
    SSD1306_WHITE
  );

  // Title
  display.setTextSize(1);
  display.setCursor(5, 0);

  display.println(
    "SMART HVAC MONITOR"
  );
```

```
// Temperature
display.setTextSize(1);
display.setCursor(0, 16);

display.print(
  "Temperature:"
);

display.setTextSize(2);
display.setCursor(65, 13);

display.print(
  temperature,
  1
);

display.print("C");

// Humidity
display.setTextSize(1);
display.setCursor(0, 37);

display.print(
  "Humidity:"
);
```

```
display.setTextSize(2);
display.setCursor(65, 34);

display.print(
  humidity,
  1
);

display.print("%");

// HVAC status
display.setTextSize(1);
display.setCursor(0, 56);

display.print("HVAC: ");

if (hvacState)
{
  display.print("ON");
}
else
{
  display.print("OFF");
}

display.display();
```

```
}

//
==========================================================
===
// 9. SEND DATA TO THINGSPEAK
//
==========================================================
===

void sendToThingSpeak(
  float temperature,
  float humidity
)
{
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println(
      "WiFi not connected!"
    );

    return;
  }

  // Create ThingSpeak URL
  String url =
    String(THINGSPEAK_SERVER) +
```

```
  "?api_key=" +
  String(THINGSPEAK_API_KEY) +
  "&field1=" +
  String(temperature, 2) +
  "&field2=" +
  String(humidity, 2) +
  "&field3=" +
  String(hvacState ? 1 : 0);

Serial.println();
Serial.println(
  "Sending data to ThingSpeak..."
);

HTTPClient http;

http.begin(url);

int httpResponseCode =
  http.GET();

if (httpResponseCode > 0)
{
  Serial.print(
    "ThingSpeak Response: "
  );
```

```
  Serial.println(
    httpResponseCode
  );

  String response =
    http.getString();

  Serial.print(
    "Entry ID: "
  );

  Serial.println(
    response
  );

  if (httpResponseCode == 200)
  {
    Serial.println(
      "ThingSpeak update successful!"
    );
  }
}
else
{
  Serial.print(
```

```
      "ThingSpeak Error: "
    );

    Serial.println(
      httpResponseCode
    );
  }

  http.end();
}

//
================================================================
===
// 10. READ DHT22
//
================================================================
===

void readSensor()
{
  float humidity =
    dht.readHumidity();

  float temperature =
    dht.readTemperature();
```

```
  // Check DHT22
  if (
    isnan(humidity) ||
    isnan(temperature)
  )
  {
    Serial.println();
    Serial.println(
      "ERROR: DHT22 reading failed!"
    );

    return;
  }

  //
==========================================================
=
  // SERIAL MONITOR
  //
==========================================================
=

  Serial.println();
  Serial.println(
    "======================================="
  );
```

```
Serial.println(
  "       SMART HVAC DATA"
);

Serial.println(
  "========================================="
);

Serial.print(
  "Temperature : "
);

Serial.print(
  temperature,
  2
);

Serial.println(" C");

Serial.print(
  "Humidity    : "
);

Serial.print(
  humidity,
  2
```

```
  );

  Serial.println(" %");

  Serial.print(
    "HVAC Status : "
  );

  if (hvacState)
  {
    Serial.println("ON");
  }
  else
  {
    Serial.println("OFF");
  }

  Serial.println(
    "=========================================="
  );

  //
==========================================================
=
  // OLED
```

```
  //
==========================================================
=

  updateOLED(
    temperature,
    humidity
  );

  //
==========================================================
=
  // THINGSPEAK
  //
==========================================================
=

  sendToThingSpeak(
    temperature,
    humidity
  );
}

//
==========================================================
===
// 11. SETUP
```

```
//
==========================================================
===

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println(
    "========================================="
  );

  Serial.println(
    " REMOTE HVAC MONITORING SYSTEM"
  );

  Serial.println(
    "========================================="
  );

  //
==========================================================
=
  // RELAY
```

```
  //
==========================================================
=

  pinMode(
    RELAY_PIN,
    OUTPUT
  );

  // Start HVAC OFF
  digitalWrite(
    RELAY_PIN,
    RELAY_OFF
  );

  hvacState = false;

  //
==========================================================
=
  // DHT22
  //
==========================================================
=

  dht.begin();
```

```
  //
==========================================================
=
  // OLED
  //
==========================================================
=

  Wire.begin(
    OLED_SDA,
    OLED_SCL
  );

  if (
    !display.begin(
      SSD1306_SWITCHCAPVCC,
      0x3C
    )
  )
  {
    Serial.println(
      "ERROR: OLED not found!"
    );

    while (true)
    {
      delay(1000);
```

```
  }
}

display.clearDisplay();

display.setTextColor(
  SSD1306_WHITE
);

display.setTextSize(1);

display.setCursor(15, 15);

display.println(
  "SMART HVAC"
);

display.setCursor(15, 30);

display.println(
  "MONITORING"
);

display.setCursor(15, 45);

display.println(
```

```
    "STARTING..."
  );

  display.display();

  delay(2000);

  //
================================================================
=
  // WIFI
  //
================================================================
=

  connectWiFi();

  Serial.println();
  Serial.println(
    "System Ready!"
  );
}

//
================================================================
===
// 12. LOOP
```

```
//
================================================================
===

void loop()
{
  // Check WiFi
  if (
    WiFi.status() != WL_CONNECTED
  )
  {
    Serial.println(
      "WiFi disconnected!"
    );

    connectWiFi();
  }

  //
================================================================
=
  // READ SENSOR EVERY 15 SECONDS
  //
================================================================
=

  unsigned long currentMillis =
    millis();
```

```
  if (
    currentMillis - previousMillis >=
    UPDATE_INTERVAL
  )
  {
    previousMillis =
      currentMillis;

    readSensor();
  }
}
```

9. System Working

The proposed HVAC monitoring system works by continuously collecting environmental and electrical parameters from sensors connected to an ESP32 microcontroller. The temperature and humidity sensor measures the surrounding conditions, while additional sensors can monitor air quality, pressure, and energy consumption. The ESP32 processes the sensor readings and connects to the internet through Wi-Fi.

The processed data is transmitted using the MQTT protocol to an MQTT broker and then forwarded to ThingSpeak Cloud for storage and visualization. ThingSpeak displays the collected parameters in real-time graphs and dashboards, allowing users to monitor HVAC performance remotely through a web browser or mobile device.

The system also compares the measured parameters with predefined threshold values. If any parameter exceeds its specified limit, the system can generate an alert and, when control functionality is implemented, activate or deactivate the HVAC equipment through a relay or suitable control interface. This continuous monitoring and feedback process helps improve energy efficiency, HVAC performance, fault detection, and user comfort.

10. Bill of Materials

S.No.	Component	Specification	Qty.	Purpose
1	ESP32 Development Board	Wi-Fi + Bluetooth	1	Main controller and MQTT communication
2	DHT22 Sensor	Temperature & Humidity	1	Measures temperature and humidity
3	MQ-135 Sensor	Air Quality	1	Detects air-quality variations
4	BMP280 Sensor	Pressure Sensor, I^2C	1	Measures atmospheric pressure
5	PZEM-004T	AC Energy Monitoring Module	1	Measures voltage, current, power and energy
6	1-Channel Relay Module	5 V	1	HVAC ON/OFF control
7	MQTT Broker	Mosquitto / Cloud MQTT	1	Handles MQTT message communication

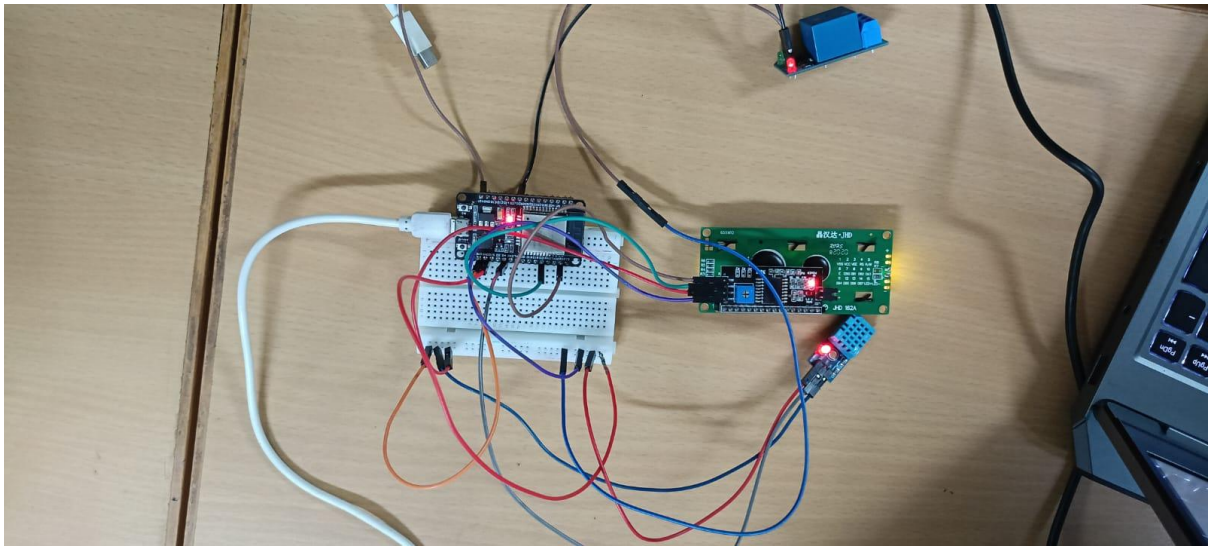
S.No.	Component	Specification	Qty.	Purpose
8	ThingSpeak Cloud	IoT Cloud Platform	1	Data storage, visualization and analysis
9	Wi-Fi Router/Hotspot	2.4 GHz	1	Internet connectivity
10	Breadboard	Standard	1	Circuit prototyping
11	Jumper Wires	Male–Male / Male–Female	1 Set	Circuit connections
12	DC Power Supply	5 V / suitable rating	1	Powers the controller and sensors
13	HVAC Load	Fan / AC / Chiller prototype	1	System under monitoring and control
14	Resistors	4.7 kΩ / 10 kΩ	As required	Pull-up and signal conditioning
15	Connecting Cables	Suitable gauge	As required	Power and sensor connections

11. Prototype Implementation

The prototype is implemented using an ESP32 microcontroller, which acts as the central processing and communication unit. Sensors such as the DHT22, MQ-135, BMP280, and PZEM-004T are connected to the ESP32 to measure temperature, humidity, air quality, pressure, and electrical energy parameters. The ESP32 collects the sensor readings at regular intervals, processes the data, and checks whether the measured values are within the predefined operating limits.

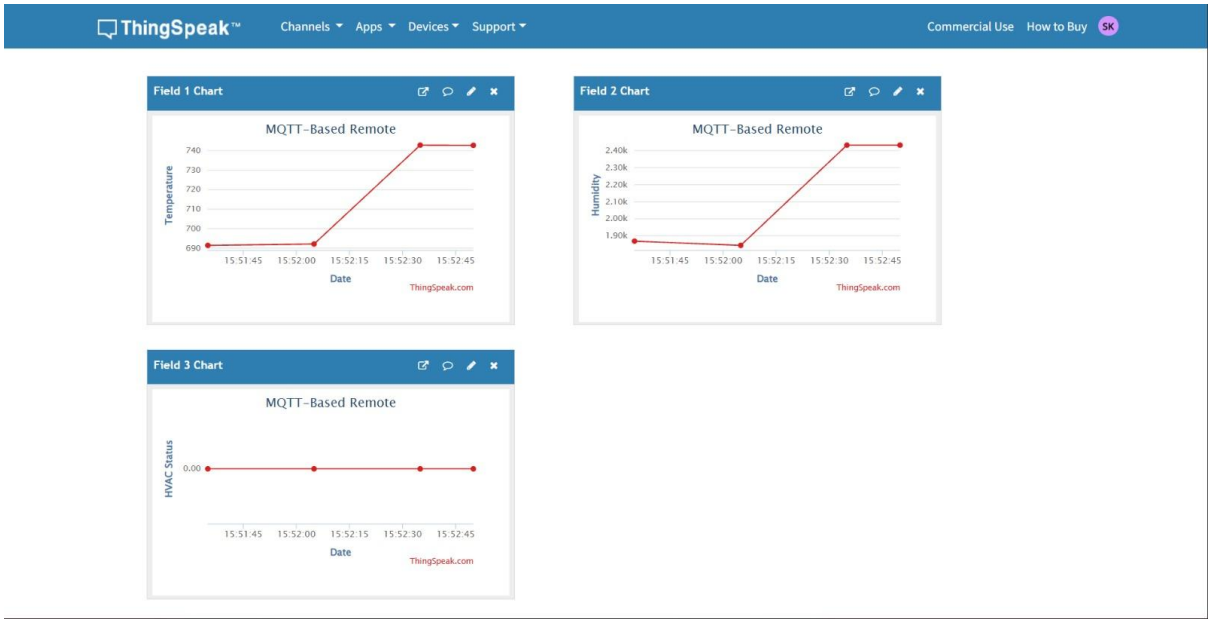
The ESP32 is connected to the internet through Wi-Fi and publishes the collected sensor data using the MQTT protocol. An MQTT broker manages the communication between the ESP32 and the cloud platform. The received data is transferred to ThingSpeak Cloud, where it is stored and displayed using real-time graphs and dashboards. This allows the user to remotely observe HVAC operating conditions from a computer or mobile device.

For control functionality, a relay module can be connected to the ESP32 to control the HVAC equipment or a prototype fan. When a parameter exceeds its defined threshold, the system can generate an alert and perform a programmed control action. The prototype is tested under different operating conditions to verify sensor accuracy, MQTT communication, cloud data transmission, real-time monitoring, and HVAC control response. This implementation demonstrates the feasibility of a low-cost IoT-based solution for remote HVAC monitoring and management.



12. Results & Performance Analysis

The developed MQTT-based remote HVAC monitoring system successfully monitored important parameters such as temperature, humidity, air quality, pressure, and energy consumption using sensors connected to the ESP32 microcontroller. The collected data was transmitted through Wi-Fi using the MQTT protocol and stored on ThingSpeak Cloud for real-time monitoring and analysis. The ThingSpeak dashboard provided graphical visualization of the sensor readings, making it easy to observe changes in HVAC operating conditions. During prototype testing, the system demonstrated reliable sensor data collection, stable MQTT communication, successful cloud data transmission, and effective remote monitoring. The threshold-based control mechanism also helped identify abnormal conditions and activate the HVAC control system when required. Overall, the prototype achieved its main objectives of real-time HVAC monitoring, cloud-based data storage, remote access, and improved system control, providing a foundation for better energy management and HVAC performance.


ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
SK
Field 1 Chart
MQTT-Based Remote
Temperature
740
730
720
710
700
690
15:51:45
15:52:00
15:52:15
15:52:30
15:52:45
Date
ThingSpeak.com
Field 2 Chart
MQTT-Based Remote
Humidity
2.40k
2.30k
2.20k
2.10k
2.00k
1.90k
15:51:45
15:52:00
15:52:15
15:52:30
15:52:45
Date
ThingSpeak.com
Field 3 Chart
MQTT-Based Remote
HVAC Status
0.00
15:51:45
15:52:00
15:52:15
15:52:30
15:52:45
Date
ThingSpeak.com